“Complaint Tracker: A Database-Driven System for Efficient Customer Complaint Management”

Project Report submitted for the partial fulfilment of grade for the course

Database Management Systems

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

SANTOSH KUMAR BAL : 2520030474

U.ANKIT : 2520030403

G.BHARGAV : 2520030451

Team Number:11

Section:2

Under the Guidance of

Dr. S. Vinay Kumar

Associate Professor



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"Complaint Tracker: A Database-Driven System for Efficient Customer Complaint Management"** is a record of Bonafide work of **SANTOSH KUMAR BAL : 2520030474 , U.ANKIT : 2520030403 ,G.BHARGAV : 2520030451**

submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

SANTOSH KUMAR BAL : 2520030474

U.ANKIT : 2520030403

G.BHARGAV : 2520030451

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is certify that the mini project based report entitled **"Complaint Tracker: A Database-Driven System for Efficient Customer Complaint Management"** is a bonafide work done and submitted by **SANTOSH KUMAR BAL : 2520030474 , U.ANKIT : 2520030403 , G.BHARGAV : 2520030451**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27.**

Signature of the GuideSignature of the HOD

(Dr. S. Vinay Kumar)

Signature of the External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his valuable guidance, constant encouragement and timely suggestions throughout the development of this mini project. His insights into database design and query optimization helped us shape the Complaint Tracker into a well-structured and reliable system.

We are thankful to Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the facilities, support and academic environment needed to complete this work successfully.

We also thank all the faculty members of the department, our friends and our families for their continuous support, motivation and cooperation during the course of this project.

SANTOSH KUMAR BAL : 2520030474

U.ANKIT : 2520030403

G.BHARGAV : 2520030451

Abstract

The Complaint Tracker is a database-driven application developed to manage customer complaints efficiently and systematically. In many organizations, complaints are handled manually through emails, spreadsheets, and paper records, which can lead to lost information, delays in resolution, and difficulties in tracking complaint status. To overcome these problems, the proposed system provides a centralized platform for recording, organizing, and monitoring customer complaints.

The system stores customer details, complaint descriptions, categories, priorities, staff assignments, and complaint status in a structured relational database. It allows users to register complaints, search and filter records, track status updates, and monitor the progress of complaint resolution. SQL queries are used to retrieve information and generate reports about complaint categories, resolution times, and recurring issues. The system also supports staff accountability by assigning complaints to responsible personnel and maintaining a history of status changes.

The main objective of the Complaint Tracker is to reduce manual work, improve data accuracy, prevent duplicate records, and support faster complaint resolution. Overall, this project provides an organized, reliable, and scalable solution for customer complaint management and helps organizations improve service quality and customer satisfaction.

Table of Contents

		Page No.
	Acknowledgement	iv
	Abstract	v
	Table of Contents	vi
	List of Figures	vii
1	**Introduction** 1.1 Overview 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Existing versus Proposed System	1-3
2	**System Requirements** 2.1 Functional Requirements 2.2 Non-Functional Requirements 2.3 Hardware and Software Requirements	4-5
3	**System Design** 3.1 High-level Architecture Diagram 3.2 Description of each Layer 3.3 Database Design 3.4 Data Dictionary 3.5 Normalization 3.6 Complaint Resolution Workflow	6-10
4	**Implementation** 4.1 Module-wise Implementation	11-16

	4.2 Front-end Logic 4.3 Application and Database Integration 4.4 Database Implementation 4.5 SQL Queries for Search and Reports 4.6 Data Integrity, Duplicate Prevention and Accountability 4.7 Role-based Access	
5	**Results** 5.1 Testing Approach 5.2 Test Cases 5.3 Output Screens 5.4 Reports Generated 5.5 Analysis of Results	17-20
6	**Challenges & Limitations** 6.1 Issues Faced during Development 6.2 Solutions Applied 6.3 Limitations of the System	21-22
7	**Conclusion & Future Work** 7.1 Conclusion 7.2 Future Work 7.3 Skills Learned during Development	23
	References	24

List of Figures

1.1	Overview of the Complaint Tracker system	1
3.1	Three-tier architecture of the Complaint Tracker	6
3.2	Entity-relationship diagram of the Complaint Tracker	7
3.3	Complaint resolution workflow	10
3.4	Complaint status lifecycle	10
5.1	Complaint registration screen	18
5.2	Complaint list with search and filter	19
5.3	Complaint details with status history	19
5.4	Reports screen	19

1. INTRODUCTION

1.1 Overview

Customer complaints are one of the most direct sources of feedback available to any organization. Banks, telecom operators, hospitals, colleges, retailers and utility providers all receive a continuous stream of complaints through calls, emails, web forms and walk-ins. How quickly and fairly these complaints are handled strongly influences customer satisfaction and the reputation of the organization.

The **Complaint Tracker** is a relational database application designed to capture, classify, assign and monitor customer complaints from the moment they are submitted until they are resolved. The system logs the details of each complaint, links it to the customer who raised it, the category it belongs to and the support staff member responsible for it, and maintains a complete status history so that every complaint can be traced at any point in its lifecycle.

As shown in Fig. 1.1, customers, support staff and administrators interact with the Complaint Tracker, which in turn stores and retrieves all information from a single relational database. Structured Query Language (SQL) is used for fast retrieval, filtering by status, category or priority, and for generating summary reports on complaint categories, resolution times and recurring issues for management review.

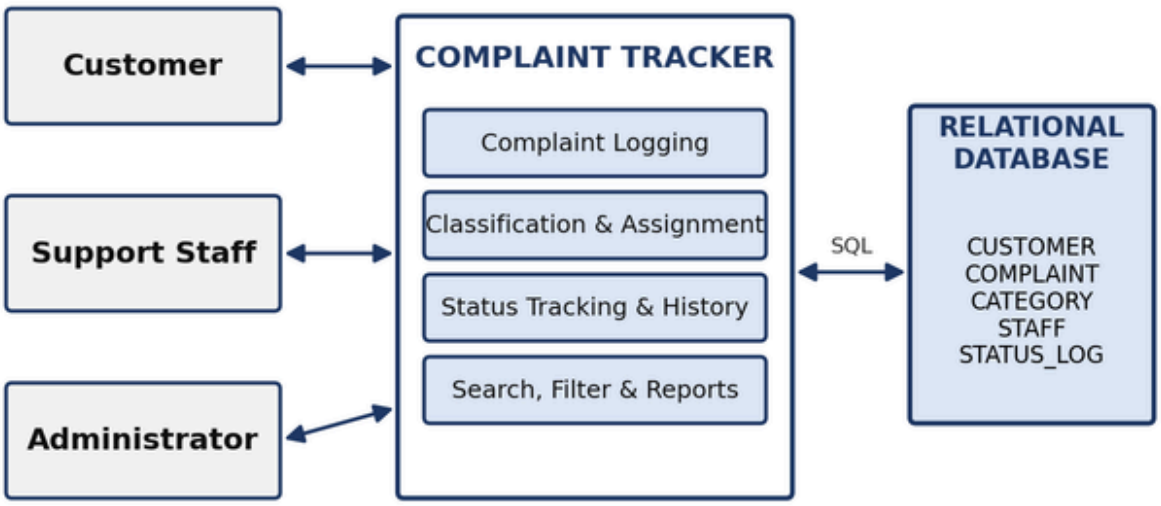


Fig. 1.1 Overview of the Complaint Tracker system

By centralizing complaint data in a normalized schema, the system reduces duplication, enforces accountability through staff assignment and shortens resolution time. The result is a reliable and scalable foundation that helps an organization turn complaint handling from a reactive burden into a measurable, continuously improving service process.

1.2 Problem Statement

In many organizations complaints are still handled manually through emails, spreadsheets and paper records. This approach does not scale as the number of complaints grows and leads to the following problems:

- **Scattered records:** complaints are logged across spreadsheets, emails and paper forms, with no single source of truth.
- **Slow resolution:** without tracking of status or ownership, complaints stall and nobody is accountable for follow-up.
- **Lost and duplicate complaints:** manual entry and repeated reporting cause records to be misplaced, duplicated or never addressed.
- **No visibility:** management cannot see trends, repeat issues or staff performance without a structured report.
- **No customer history:** there is no linkage between a customer and the complaints raised earlier, so recurring problems go unnoticed.

There is therefore a need for a centralized, database-driven system that records every complaint in a structured form, assigns it to a responsible person, tracks each change of status and produces reliable reports on demand.

1.3 Scope of the Project

The scope of the Complaint Tracker covers the complete lifecycle of a customer complaint inside an organization. The project includes:

- Registration and storage of customer details and complaint details in a normalized relational database.
- Classification of complaints by category and priority.
- Assignment of each complaint to a responsible staff member or department.
- Recording of every status change with remarks, staff identity and date and time, so that a full history is available.
- Searching and filtering of complaints by status, category, priority, date and customer.
- Generation of SQL-based reports on complaint categories, resolution times, staff workload and recurring issues.
- Prevention of duplicate and inconsistent records through keys, constraints and validation.

The project is limited to a single-organization deployment. Automated text classification, a native mobile application and predictive analytics are outside the present scope and are listed as future enhancements in Chapter 7.

1.4 Objectives

The main objective of the Complaint Tracker is to reduce manual work, improve data accuracy, prevent duplicate records and support faster complaint resolution. The specific objectives are:

- **Centralize data:** store all complaint records in one normalized relational database.
- **Enable fast retrieval:** support quick search and filtering by status, category and customer.
- **Assign accountability:** route each complaint to a responsible staff member and keep a history of every status change.
- **Track performance:** generate reports on resolution time and recurring issue trends.
- **Improve quality:** reduce manual effort and data errors so that the organization can improve service quality and customer satisfaction.

1.5 Existing versus Proposed System

Table 1.1 compares the manual method commonly used at present with the proposed Complaint Tracker.

Table 1.1 Existing system versus proposed system

Aspect	Existing (Manual) System	Proposed (Complaint Tracker)
Storage	Spreadsheets, e-mails or paper logs	All complaints stored in a normalized relational database
Customer history	No linkage between a customer and earlier complaints	Complete complaint history of a customer is one query away
Status and ownership	Manual updates, easy to forget or lose	Status and staff assignment tracked automatically with history
Reporting	Compiled by hand, slow and error-prone	Instant SQL-based reports and summaries
Duplicates	Frequent duplicate or missing records	Keys, constraints and validation prevent duplicate records

2. SYSTEM REQUIREMENTS

This chapter lists what the Complaint Tracker must do (functional requirements), the qualities it must have (non-functional requirements) and the hardware and software needed to build and run it.

2.1 Functional Requirements

Three types of users interact with the system: the **customer**, who submits complaints and follows their progress; the **support staff member**, who investigates and updates complaints; and the **administrator**, who manages master data, assigns work and reviews reports. Table 2.1 lists the functional requirements.

Table 2.1 Functional requirements

ID	Requirement	Description
FR-01	Customer registration	Store customer name, e-mail, phone and address; a customer is identified by a unique customer ID.
FR-02	Complaint registration	Record a complaint with description, category, priority and date filed, linked to the customer.
FR-03	Classification	Assign every complaint to a category and a priority level (for example Low, Medium, High, Critical).
FR-04	Staff assignment	Route each complaint to a responsible staff member or department so that ownership is always known.
FR-05	Status update	Allow staff to update status (Submitted, Assigned, In Progress, Resolved, Closed) with remarks.
FR-06	Status history	Keep a time-stamped log of every status change, with the staff member who made it.
FR-07	Search and filter	Search complaints by customer, category, status, priority and date range.
FR-08	Customer history	Display all complaints raised by a given customer.
FR-09	Reports	Generate reports on complaints per category, average resolution time, staff workload and recurring issues.
FR-10	Duplicate prevention	Detect and block duplicate customers and duplicate complaint entries.
FR-11	Notifications	Inform customers and staff when the status of a complaint changes.
FR-12	Role-based access	Restrict operations according to the role of the user (customer, staff, administrator).

2.2 Non-Functional Requirements

Table 2.2 Non-functional requirements

Attribute	Requirement
Data integrity	Primary keys, foreign keys, NOT NULL, UNIQUE and CHECK constraints must keep the data consistent at all times.
Accuracy	Validation of input must reduce data-entry errors and duplicate records.
Performance	Search, filter and report queries should return results quickly; frequently searched columns are indexed.
Reliability	No complaint record may be lost; status updates must be stored atomically so that history is never partially written.
Security	Access is controlled by role; personal details of customers are visible only to authorized users.
Usability	Simple forms and clear screens so that staff can log and update a complaint in a few steps.
Scalability	The normalized schema must handle a growing number of customers and complaints without redesign.
Maintainability	Modular design (presentation, application and database layers) so that each part can be changed independently.

2.3 Hardware and Software Requirements

Table 2.3 Hardware requirements

Component	Minimum requirement
Processor	Intel Core i3 or equivalent
Memory (RAM)	4 GB (8 GB recommended)
Storage	500 MB free disk space for the application and database
Display	1366 x 768 resolution or higher

Table 2.4 Software requirements

Component	Technology
Operating system	Windows 10 / 11, Linux or macOS
Front end	HTML, CSS, JavaScript / React
Back end	Python (Django) or Java (Spring)
Database	MySQL / PostgreSQL
Development tool	Visual Studio Code
Web browser	Google Chrome, Microsoft Edge or Mozilla Firefox (latest version)
Hosting	Cloud server or on-premise server

3. SYSTEM DESIGN

This chapter presents the design of the Complaint Tracker: its three-tier architecture, the database schema with its entity-relationship model, the normalization of the schema and the workflow followed by a complaint.

3.1 High-level Architecture Diagram

The Complaint Tracker follows a three-tier architecture, shown in Fig. 3.1. Users work with the presentation layer, which sends requests to the application layer. The application layer applies the business rules and communicates with the database layer using SQL queries. Keeping the layers separate means that the user interface, the business logic and the stored data can each be modified without disturbing the others.

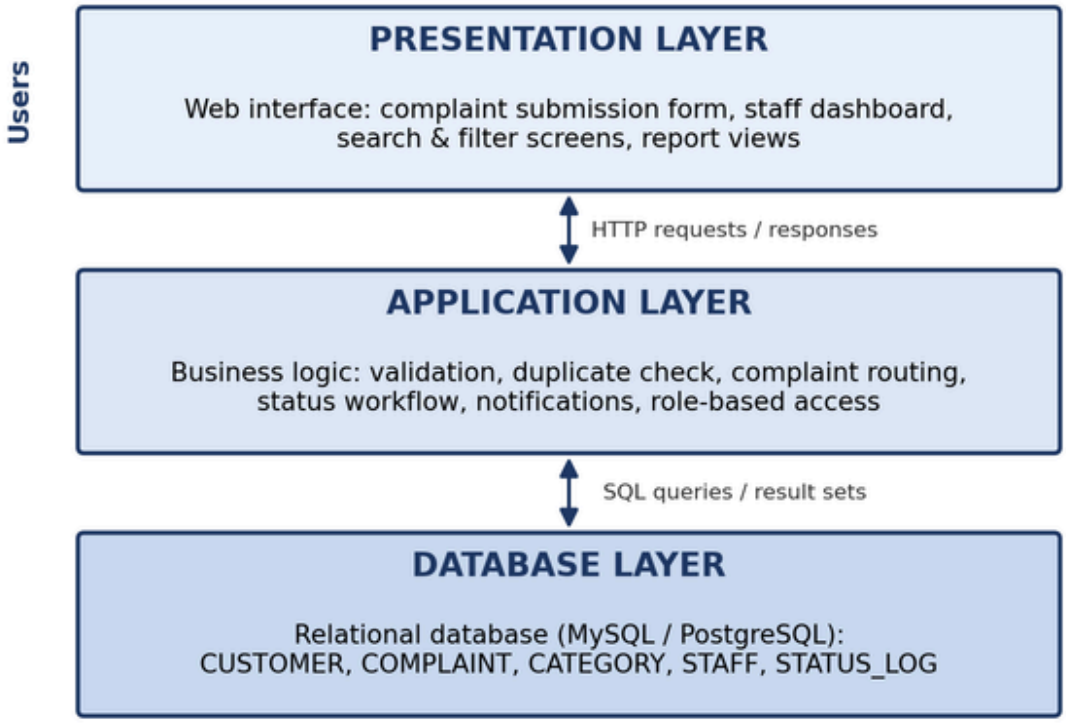


Fig. 3.1 Three-tier architecture of the Complaint Tracker

3.2 Description of each Layer

Presentation Layer

This is the web or desktop interface through which customers submit complaints and staff manage cases. It contains the complaint registration form, the staff dashboard, the search and filter screens and the report views. It validates simple input errors, such as empty fields or an invalid e-mail format, before the data is sent on.

Application Layer

This layer holds the business logic of the system: validation of requests, duplicate checking, routing of a complaint to the right staff member or department, enforcement of the status workflow, generation of notifications and checking of the role of the user. It is the only layer that talks to the database.

Database Layer

The database layer is a relational database (MySQL / PostgreSQL) that stores customers, complaints, categories, staff and status logs. It enforces integrity through primary keys, foreign keys and constraints, and answers the

SQL queries sent by the application layer for retrieval and reporting.

3.3 Database Design

The database is designed around five core entities: CUSTOMER, COMPLAINT, CATEGORY, STAFF and STATUS_LOG. The entity-relationship diagram is shown in Fig. 3.2. The relationships are:

- One **customer** can file many **complaints**; each complaint is filed by exactly one customer (1 : N).
- One **category** can classify many **complaints**; each complaint belongs to exactly one category (1 : N).
- One **complaint** has many **status log** entries, which together form its history (1 : N).
- One **staff** member can make many **status log** entries; each entry records the staff member responsible for that update (1 : N).

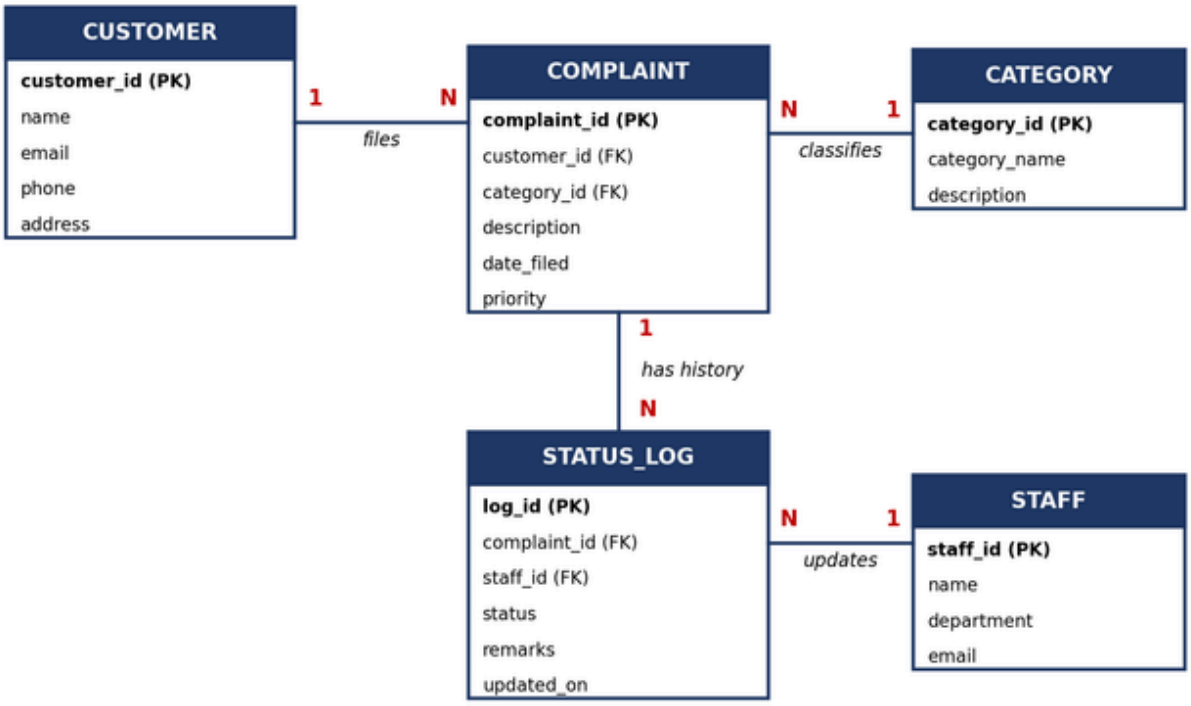


Fig. 3.2 Entity-relationship diagram of the Complaint Tracker

Staff assignment is stored in STATUS_LOG: the first entry that names a staff member records the assignment, and the most recent entry for a complaint gives its current status and current owner. Because every change adds a new row instead of overwriting an old one, the complete history of a complaint is preserved.

3.4 Data Dictionary

Tables 3.1 to 3.5 describe the columns of each table. Primary keys are marked PK and foreign keys FK.

Table 3.1 CUSTOMER

Column	Data type	Constraint	Description
customer_id	INT	PK, AUTO_INCREMENT	Unique identifier of the customer
name	VARCHAR(100)	NOT NULL	Full name of the customer
email	VARCHAR(100)	NOT NULL, UNIQUE	E-mail address; prevents duplicate customers
phone	VARCHAR(15)		Contact number
address	VARCHAR(255)		Postal address

Table 3.2 CATEGORY

Column	Data type	Constraint	Description
category_id	INT	PK, AUTO_INCREMENT	Unique identifier of the category
category_name	VARCHAR(60)	NOT NULL, UNIQUE	Name such as Billing, Service, Product
description	VARCHAR(255)		What kind of complaints belong here

Table 3.3 STAFF

Column	Data type	Constraint	Description
staff_id	INT	PK, AUTO_INCREMENT	Unique identifier of the staff member
name	VARCHAR(100)	NOT NULL	Name of the staff member
department	VARCHAR(60)	NOT NULL	Department the staff member belongs to
email	VARCHAR(100)	NOT NULL, UNIQUE	Official e-mail address

Table 3.4 COMPLAINT

Column	Data type	Constraint	Description
complaint_id	INT	PK, AUTO_INCREMENT	Unique complaint number
customer_id	INT	FK, NOT NULL	References CUSTOMER(customer_id)
category_id	INT	FK, NOT NULL	References CATEGORY(category_id)
description	TEXT	NOT NULL	Details of the complaint
date_filed	DATE	NOT NULL	Date on which the complaint was filed
priority	VARCHAR(10)	NOT NULL, CHECK	Low, Medium, High or Critical

Table 3.5 STATUS_LOG

Column	Data type	Constraint	Description
log_id	INT	PK, AUTO_INCREMENT	Unique identifier of the log entry
complaint_id	INT	FK, NOT NULL	References COMPLAINT(complaint_id)
staff_id	INT	FK, NOT NULL	References STAFF(staff_id); staff member responsible
status	VARCHAR(20)	NOT NULL, CHECK	Submitted, Assigned, In Progress, Resolved, Closed
remarks	VARCHAR(255)		Notes about the action taken
updated_on	DATETIME	NOT NULL	Date and time of the update

3.5 Normalization

The schema has been normalized up to the Third Normal Form (3NF) to avoid redundancy and update anomalies.

- **First Normal Form (1NF):** every column holds a single atomic value and every table has a primary key. For example, a customer's phone number and address are stored in separate columns, and a complaint's history is stored as separate rows in STATUS_LOG instead of a repeating group inside COMPLAINT.
- **Second Normal Form (2NF):** every non-key column depends on the whole primary key. All tables use a single-column key, so no partial dependency can occur.
- **Third Normal Form (3NF):** no non-key column depends on another non-key column. Customer details are kept in CUSTOMER, category details in CATEGORY and staff details in STAFF; COMPLAINT and STATUS_LOG store only their foreign keys, so a change of a customer's e-mail or a staff member's department is made in exactly one place.

3.6 Complaint Resolution Workflow

Every complaint moves through the five stages shown in Fig. 3.3. The complaint is submitted, classified and assigned, investigated by the responsible staff member, updated as work progresses and finally resolved and closed. Each transition adds a row to STATUS_LOG.



Fig. 3.3 Complaint resolution workflow

The allowed values of the status field and the transitions between them are shown in Fig. 3.4. A resolved complaint can be reopened and re-assigned if the customer is not satisfied with the outcome.

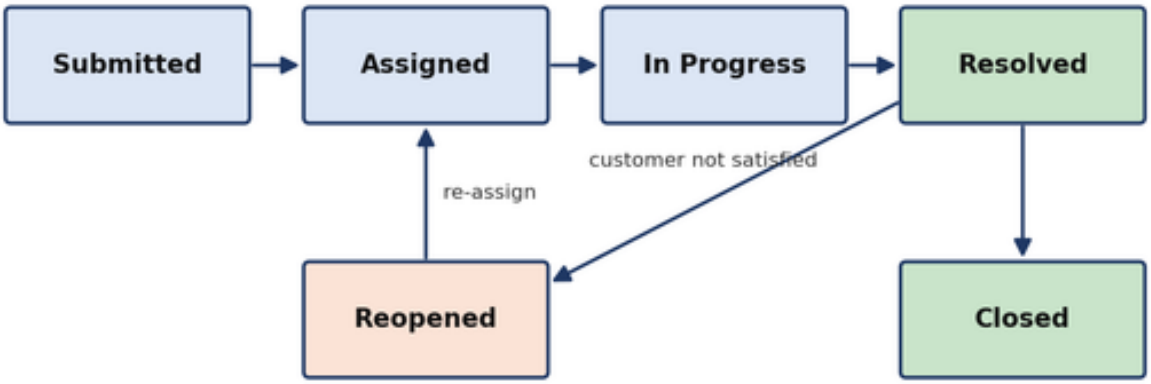


Fig. 3.4 Complaint status lifecycle

4. IMPLEMENTATION

This chapter describes how the design of Chapter 3 is realized: the modules of the system, the front-end logic, the interaction between the application and the database, the SQL used to create the database and generate reports, and the measures taken to keep the data correct and secure. The project was developed and tested in Visual Studio Code.

4.1 Module-wise Implementation

The Complaint Tracker is divided into the modules summarized in Table 4.1.

Table 4.1 Modules of the Complaint Tracker

Module	Function	Tables used
Customer Management	Adds, updates and searches customer records; shows the complaint history of a customer.	CUSTOMER, COMPLAINT
Complaint Logging	Quick entry of a complaint with category, priority and customer linkage; blocks duplicates.	COMPLAINT, CUSTOMER, CATEGORY
Classification and Assignment	Routes the complaint to the right staff member or department and records the assignment.	COMPLAINT, STAFF, STATUS_LOG
Status Tracking	Updates the status with remarks and keeps a time-stamped history of every change.	STATUS_LOG
Search and Filter	Finds complaints by date, status, category, priority or customer.	All tables
Reports and Analytics	Produces summaries of complaints per category, resolution time, staff workload and recurring issues.	All tables
Access Control	Applies role-based permissions for customer, staff and administrator.	STAFF

Customer Management

A customer is registered once and identified by customer_id. The UNIQUE constraint on the e-mail column prevents the same customer from being stored twice. When a complaint is entered, the application first looks up the customer; if the customer exists, the new complaint is linked to the existing record, which keeps the complaint history of that customer complete.

Complaint Logging

The complaint form collects the customer, category, priority and description. The date_filed is set automatically. Before the complaint is inserted, a duplicate check searches for an identical open complaint from the same customer in the same category (see Section 4.6). A new complaint immediately receives its first STATUS_LOG row with the status Submitted.

Classification and Assignment

Each category is handled by a department. The application layer selects a staff member of that department, for example the one with the fewest open complaints, and writes a STATUS_LOG row with the status Assigned and

the selected staff_id. This gives every complaint a responsible owner from the start.

Status Tracking

When a staff member changes the status, a new row is added to STATUS_LOG with the new status, remarks, the staff_id and the current date and time. Old rows are never edited, so the complete audit trail remains available. The insertion is done inside a database transaction so that a status change is either stored completely or not at all.

Search, Filter and Reports

Search screens build SQL queries with WHERE conditions on status, category, priority, date and customer. The report module uses aggregate functions such as COUNT and AVG with GROUP BY, as shown in Section 4.5.

4.2 Front-end Logic

The presentation layer is built with HTML, CSS and JavaScript (React can be used for component-based rendering). Its main responsibilities are:

- **Forms and validation:** the complaint and customer forms check for empty required fields, valid e-mail format and phone number length before submission, and show clear error messages.
- **Rendering of lists:** complaint lists are displayed in tables showing complaint number, customer, category, priority, current status and date filed, with colour-coded priority and status labels.
- **Search and filter controls:** drop-downs for status, category and priority and a date range selector send the chosen criteria to the application layer and re-render the result list.
- **Status timeline:** the detail screen of a complaint shows its STATUS_LOG entries in order, so that the user can see who did what and when.
- **State management:** the selected filters, the open complaint and the logged-in role are held in the state of the interface so that screens update without reloading the whole page.

4.3 Application and Database Integration

The application layer receives each request from the interface, checks the role of the user, validates the data and then runs the corresponding SQL statement on the database. Results are returned to the interface for display. Table 4.2 lists the main operations and the type of SQL statement behind each one.

Table 4.2 Main operations of the application layer

Operation	SQL statement type	Purpose
Register customer	INSERT into CUSTOMER	Create a new customer record
Log complaint	INSERT into COMPLAINT and STATUS_LOG	Create the complaint and its first status entry
Assign complaint	INSERT into STATUS_LOG	Record the responsible staff member
Update status	INSERT into STATUS_LOG (transaction)	Add a new history entry with remarks
Search complaints	SELECT with JOIN and WHERE	Filter by status, category, priority, date or customer
Customer history	SELECT with JOIN	List all complaints of one customer
Generate report	SELECT with GROUP BY and aggregates	Counts, averages and recurring issues

Queries that use user input are written as parameterized (prepared) statements, so that input is never joined directly into the SQL text. This protects the database from SQL injection.

4.4 Database Implementation

The database is created with the following SQL script. Foreign keys connect COMPLAINT to CUSTOMER and CATEGORY, and STATUS_LOG to COMPLAINT and STAFF. CHECK constraints restrict priority and status to valid values, and indexes speed up the most common searches.

Listing 4.1 Table creation script

```
CREATE TABLE customer (

    customer_id  INT AUTO_INCREMENT PRIMARY KEY,

    name         VARCHAR(100) NOT NULL,

    email        VARCHAR(100) NOT NULL UNIQUE,

    phone        VARCHAR(15),

    address      VARCHAR(255)

);
```

```
CREATE TABLE category (
    category_id    INT AUTO_INCREMENT PRIMARY KEY,
    category_name  VARCHAR(60) NOT NULL UNIQUE,
    description    VARCHAR(255)
);

CREATE TABLE staff (
    staff_id    INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    department  VARCHAR(60) NOT NULL,
    email       VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE complaint (
    complaint_id  INT AUTO_INCREMENT PRIMARY KEY,
    customer_id   INT NOT NULL,
    category_id   INT NOT NULL,
    description   TEXT NOT NULL,
    date_filed    DATE NOT NULL,
    priority      VARCHAR(10) NOT NULL
                  CHECK (priority IN ('Low','Medium','High','Critical')),
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id),
    FOREIGN KEY (category_id) REFERENCES category(category_id)
);

CREATE TABLE status_log (
    log_id        INT AUTO_INCREMENT PRIMARY KEY,
    complaint_id  INT NOT NULL,
    staff_id      INT NOT NULL,
    status        VARCHAR(20) NOT NULL
                  CHECK (status IN ('Submitted','Assigned','In Progress',
                                    'Resolved','Closed','Reopened')),
    remarks       VARCHAR(255),
```

```
    updated_on    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (complaint_id) REFERENCES complaint(complaint_id),
    FOREIGN KEY (staff_id)     REFERENCES staff(staff_id)
);

CREATE INDEX idx_complaint_customer ON complaint(customer_id);
CREATE INDEX idx_complaint_category ON complaint(category_id);
CREATE INDEX idx_log_complaint      ON status_log(complaint_id);
```

4.5 SQL Queries for Search and Reports

The following queries implement the search and reporting features. Query 1 lists complaints of a customer with their category; Queries 2 to 5 are the management reports mentioned in the abstract.

Listing 4.2 Query 1 - complaint history of a customer

```
SELECT c.complaint_id, cat.category_name, c.priority, c.date_filed
FROM   complaint c
JOIN   category cat ON cat.category_id = c.category_id
WHERE  c.customer_id = ?
ORDER  BY c.date_filed DESC;
```

Listing 4.3 Query 2 - current status of every complaint (latest log entry)

```
SELECT c.complaint_id, cu.name AS customer, cat.category_name,
       c.priority, sl.status, s.name AS assigned_to, sl.updated_on
FROM   complaint c
JOIN   customer  cu  ON cu.customer_id  = c.customer_id
JOIN   category  cat ON cat.category_id = c.category_id
JOIN   status_log sl ON sl.complaint_id = c.complaint_id
JOIN  (SELECT MAX(log_id) AS log_id
       FROM   status_log GROUP BY complaint_id) latest
       ON latest.log_id = sl.log_id
JOIN   staff s ON s.staff_id = sl.staff_id
WHERE  sl.status = ?          -- filter by status
ORDER  BY c.date_filed;
```

Listing 4.4 Query 3 - complaints per category

```
SELECT cat.category_name, COUNT(*) AS total_complaints

FROM   complaint c

JOIN   category cat ON cat.category_id = c.category_id

GROUP  BY cat.category_name

ORDER  BY total_complaints DESC;
```

Listing 4.5 Query 4 - average resolution time per category (days)

```
SELECT cat.category_name,

       AVG(DATEDIFF(sl.updated_on, c.date_filed)) AS avg_days_to_resolve

FROM   complaint c

JOIN   category   cat ON cat.category_id = c.category_id

JOIN   status_log sl  ON sl.complaint_id = c.complaint_id

                     AND sl.status = 'Resolved'

GROUP  BY cat.category_name;
```

Listing 4.6 Query 5 - recurring issues (same customer, same category)

```
SELECT cu.name, cat.category_name, COUNT(*) AS times_reported

FROM   complaint c

JOIN   customer cu  ON cu.customer_id  = c.customer_id

JOIN   category cat ON cat.category_id = c.category_id

GROUP  BY cu.customer_id, cu.name, cat.category_id, cat.category_name

HAVING COUNT(*) >= 2

ORDER  BY times_reported DESC;
```

Staff workload is obtained in the same way: the latest STATUS_LOG row of every complaint is joined with STAFF and the complaints whose status is not Resolved or Closed are counted per staff member. This report helps the administrator balance assignments.

4.6 Data Integrity, Duplicate Prevention and Accountability

- **Referential integrity:** foreign keys make it impossible to store a complaint for a customer or category that does not exist, or a log entry for an unknown complaint or staff member.
- **Duplicate customers:** the UNIQUE constraint on CUSTOMER.email rejects a second record for the same e-mail address.

- **Duplicate complaints:** before inserting a complaint, the application runs a check for an identical open complaint of the same customer in the same category (query below). If one exists, the user is shown the existing complaint instead of creating a new one.
- **Valid values:** CHECK constraints on priority and status keep the data within the allowed values.
- **Accountability:** every STATUS_LOG row stores the staff_id and the time of the change, so it is always known who handled a complaint and when.

Listing 4.7 Duplicate-complaint check

```
SELECT c.complaint_id

FROM   complaint c

JOIN   status_log sl ON sl.complaint_id = c.complaint_id

JOIN  (SELECT MAX(log_id) AS log_id FROM status_log

       GROUP BY complaint_id) latest ON latest.log_id = sl.log_id

WHERE  c.customer_id = ? AND c.category_id = ?

  AND  sl.status NOT IN ('Resolved','Closed');
```

4.7 Role-based Access

Role-based access keeps data secure and auditable. Table 4.3 shows the permissions of each role.

Table 4.3 Permissions of each role

Operation	Customer	Staff	Administrator
Submit a complaint	Yes	Yes	Yes
View own complaints and status	Yes	Yes	Yes
View all complaints	No	Assigned / department	Yes
Update status and add remarks	No	Yes	Yes
Assign or re-assign complaints	No	No	Yes
Manage categories and staff	No	No	Yes
View reports	No	Limited	Yes

5. RESULTS

This chapter describes how the Complaint Tracker was tested and what it delivers. The system was run on a development machine using Visual Studio Code and a local relational database, and its behaviour was checked against the functional requirements of Chapter 2.

5.1 Testing Approach

- **Database testing:** the table creation script was executed and the constraints (primary key, foreign key, UNIQUE, CHECK) were tested by inserting both valid and invalid rows.
- **Functional testing:** each module (registration, logging, assignment, status update, search, report) was exercised with sample customers, categories and staff.
- **Query testing:** every search and report query was run on sample data and its output compared with a manual count.
- **Integration testing:** the complete flow from submitting a complaint to closing it was run end to end through the interface.

5.2 Test Cases

Table 5.1 Test cases and expected outcomes

ID	Scenario	Action	Expected outcome
TC-01	Register a new customer	Enter name, unique e-mail, phone, address	Customer stored with a new customer_id
TC-02	Duplicate customer	Register a second customer with the same e-mail	Rejected by the UNIQUE constraint; error shown
TC-03	Log a complaint	Choose customer, category, priority and enter description	Complaint stored; first STATUS_LOG row is Submitted
TC-04	Invalid priority	Insert priority value other than Low, Medium, High or Critical	Rejected by the CHECK constraint
TC-05	Unknown customer	Insert a complaint with a non-existent customer_id	Rejected by the foreign key constraint
TC-06	Duplicate complaint	Log the same open complaint again for the same customer and category	Existing complaint is shown; no new record
TC-07	Assign complaint	Assign a complaint to a staff member	STATUS_LOG row with status Assigned and the staff_id
TC-08	Update status	Change status to In Progress with remarks	New row added; earlier rows unchanged
TC-09	Filter by status	Select status Resolved in the filter	Only complaints whose latest status is Resolved are listed
TC-10	Customer history	Open the history of one customer	All complaints of that customer are listed
TC-11	Category report	Run the complaints-per-category report	Counts match a manual count of the data
TC-12	Resolution-time report	Run the average resolution-time report	Average days per category for resolved complaints

5.3 Output Screens

Figures 5.1 to 5.4 show the main screens of the working application.

[Insert screenshot of the complaint registration form from your running project]

Fig. 5.1 Complaint registration screen

[Insert screenshot of the complaint list with the status / category / priority filters]

Fig. 5.2 Complaint list with search and filter

[Insert screenshot of a complaint with its STATUS_LOG timeline and the status update form]

Fig. 5.3 Complaint details with status history

[Insert screenshot of the reports (complaints per category, resolution time, recurring issues)]

Fig. 5.4 Reports screen

5.4 Reports Generated

Table 5.2 Reports produced by the system

Report	Information provided	Used by
Complaints per category	Number of complaints in each category	Management
Average resolution time	Average days from filing to resolution for each category	Management
Recurring issues	Customers who reported the same category more than once	Administrator
Staff workload	Number of open complaints per staff member	Administrator
Status summary	Number of complaints in each status	Management, staff

5.5 Analysis of Results

Table 5.3 shows how the objectives set in Chapter 1 are met by the system.

Table 5.3 Objectives and how they are achieved

Objective	How it is achieved
Centralize data	All complaint, customer, category, staff and status data is stored in one normalized database.
Fast retrieval	Indexed columns and SQL filters return complaints by status, category, customer or date.
Accountability	Every complaint has an assigned staff member and every change is logged with staff and time.
Performance tracking	SQL reports show resolution time, workload and recurring issues.
Reduced manual work and errors	Constraints and validation prevent duplicate and invalid records; status history is kept automatically.

6. CHALLENGES & LIMITATIONS

6.1 Issues Faced during Development

- **Modelling the status history:** deciding whether to keep a single status column in COMPLAINT or a separate history table. A single column would overwrite earlier information and lose the audit trail.
- **Finding the current status:** because the history is stored as many rows, the latest row of each complaint has to be located for every list and filter.
- **Duplicate records:** the same customer could be entered with a slightly different spelling, and the same complaint could be reported more than once.
- **Choosing the responsible staff member:** routing complaints fairly across staff of the right department.
- **Report accuracy:** calculating resolution time correctly when a complaint is reopened or has several status changes.
- **Keeping data consistent:** a status change must not be saved partly, and no log entry may refer to a missing complaint or staff member.

6.2 Solutions Applied

Table 6.1 Issues and solutions

Issue	Solution applied
Loss of history	A separate STATUS_LOG table stores one new row for every change; rows are never overwritten.
Current status lookup	A sub-query selects MAX(log_id) for each complaint and joins it back to STATUS_LOG; an index on complaint_id keeps it fast.
Duplicate customers	UNIQUE constraint on the e-mail column and a look-up before every new registration.
Duplicate complaints	A check for an open complaint of the same customer and category before each insert.
Staff routing	Complaints are routed by category to a department and then to the staff member with the fewest open complaints.
Report accuracy	Resolution time is measured from date_filed to the Resolved entry in STATUS_LOG.
Consistency	Foreign keys, CHECK constraints and transactions for status updates.

6.3 Limitations of the System

- Complaints are categorized and prioritized by the user; there is no automatic text analysis.
- Notifications are limited to e-mail style alerts; SMS and push notifications are not provided.
- There is no native mobile application; the system is used through a web or desktop interface.
- The system is designed for a single organization and does not support several organizations or branches with separate data.
- Reports describe past data only; there is no forecasting of complaint volume or staffing needs.
- Attachments such as photos or documents are not stored with a complaint.

7. CONCLUSION & FUTURE WORK

7.1 Conclusion

The Complaint Tracker replaces scattered, manual complaint handling with a structured relational database that centralizes records, assigns accountability and provides the reports managers need to raise service quality. A normalized schema of five tables stores customers, complaints, categories, staff and the status history, while SQL queries provide fast search, filtering and reporting.

The system reduces manual work, improves the accuracy of data, prevents duplicate records and supports faster resolution of complaints, as set out in the objectives. By keeping a complete, time-stamped history of every complaint and the staff member who handled it, the system turns every complaint into an opportunity to improve the service of the organization.

7.2 Future Work

- **AI-based categorization:** automatically classify and prioritize complaints using text analysis.
- **Mobile application:** a native app for on-the-go complaint submission and tracking.
- **Predictive analytics:** forecast complaint volume and staffing needs from historical trends.
- **Multi-channel alerts:** SMS and push notifications alongside e-mail updates.
- **Attachments and escalation:** upload of supporting files and automatic escalation of complaints that stay open beyond a time limit.
- **Analytics dashboard:** richer visual dashboards for resolution time and complaint trends.

7.3 Skills Learned during Development

- Analysing a real-world problem and converting it into entities, attributes and relationships (ER modelling).
- Designing and normalizing a relational schema up to the Third Normal Form.
- Writing SQL for table creation, constraints, joins, sub-queries, aggregation and reports.
- Using primary keys, foreign keys, CHECK constraints, indexes and transactions to keep data correct.
- Developing, running and testing the application in Visual Studio Code.
- Preparing test cases, documenting the work and collaborating as a team.

REFERENCES

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.

[2] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed. McGraw-Hill Education, 2019.

[3] E. F. Codd, “A relational model of data for large shared data banks,” Communications of the ACM, vol. 13, no. 6, pp. 377–387, 1970.

[4] P. P. Chen, “The entity-relationship model – toward a unified view of data,” ACM Transactions on Database Systems, vol. 1, no. 1, pp. 9–36, 1976.

[5] MySQL 8.0 Reference Manual, Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/

[6] PostgreSQL Documentation, The PostgreSQL Global Development Group. https://www.postgresql.org/docs/

[7] Django Documentation, Django Software Foundation. https://docs.djangoproject.com/

[8] Visual Studio Code Documentation, Microsoft. https://code.visualstudio.com/docs

[9] W3Schools SQL Tutorial. https://www.w3schools.com/sql/